ASSIGNMENT 2

TOPIC: UNIFIED MODELING LANGUAGE (UML) • THEORY, TAXONOMY & APPLICATION

1. What is UML?

Unified Modeling Language (UML) is an industry-standard, standardized visual modeling language developed by Grady Booch, Ivar Jacobson, and James Rumbaugh ("The Three Amigos") and maintained by the Object Management Group (OMG). It provides software engineers, systems analysts, and architects with a standardized graphical notation to **specify, visualize, construct, and document** the artifacts of an object-oriented software system. UML is programming-language independent; it models real-world business domains, system static structures, behavioral dynamics, and physical deployment topologies before and during software development.

2. What is the use of UML?

In modern software engineering, UML serves several vital engineering purposes:

• **Architectural Blueprinting:** Provides an unambiguous visual blueprint of system components, class hierarchies, and database schemas before coding commences, mitigating costly architectural re-engineering.

• **Cross-Functional Communication:** Bridges technical developers, database administrators, academic project guides, and non-technical stakeholders through a universal graphical medium.

• **Behavioral & Workflow Modeling:** Illustrates the exact sequence of message passing, state transitions, and conditional decision branching between interacting runtime objects.

• **Formal System Documentation:** Acts as permanent, version-controlled technical documentation that accelerates project onboarding, refactoring, and future feature maintenance.

• *Application in College AI Chatbot:* UML was instrumental in mapping user roles (Student and Admin actors), structural associations between entities (Student to ChatHistory), and the asynchronous query-response lifeline execution.

3. Explain types of UML diagram with example.

UML (version 2.5) defines 14 official diagram types divided into two overarching categories:

A. Structural Diagrams (Static System Architecture):

1. *Class Diagram:* Illustrates the system's static structure by showing classes, attributes, operations (methods), and associations. Example: In our project, the Student class with attributes (roll_no, attendance, cgpa) associated in a 1-to-0..* relationship with ChatHistory.
2. *Object Diagram:* Represents a snapshot of runtime instances with assigned slot values at a specific moment in time. Example: An object instance kanishk : Student with roll_no = "ISTU25010120001" and attendance = 82.5%.
3. *Component Diagram:* Depicts how software modules, libraries, and executables are wired together via interfaces. Example: ChatbotUI component connecting via REST API to FlaskServer and SQLite3 DB.
4. *Deployment Diagram:* Visualizes the hardware execution topology and runtime artifacts. Example: User client browser hardware executing over HTTPS to Render Linux cloud servers.

B. Behavioral Diagrams (Dynamic System Execution & Interaction):

1. *Use Case Diagram:* Illustrates external actors and their observable interactions with system use cases within a defined system boundary. Example: Student executing "Ask College FAQs" and "Student Login".
2. *Sequence Diagram:* Models time-ordered message exchange between participating object lifelines. Example: Student enters voice query → UI sends POST → Flask queries SQLite → Audio synthesis returns to student.
3. *Activity Diagram:* Represents procedural workflow, control flow branching (decision diamonds), and concurrent activity forks. Example: The decision flow checking whether a user query is authenticated before executing attendance detention checks.
4. *State Machine Diagram:* Models the state transitions of an entity across its lifecycle in response to external events. Example: A query transitioning from Received → Parsing → Matched → Logged.

4. Draw UML Diagrams of your project — (a) Use Case Diagram

System Boundary & Actor Interactions: Models the functional requirements of the College AI Chatbot. The system includes two primary actors: **Student** (queries FAQs, uses voice input/TTS, logs in for attendance/CGPA, submits feedback ratings, exports chat transcripts) and **Admin** (authenticates with hashed password, views KPI analytics, updates FAQ database, and exports CSV audit logs). It incorporates the mandatory <<include>> relationship (Ask College FAQs includes querying campus address and fees) and the <<extend / exclude>> dependency (Voice Input extends the standard text query use case).

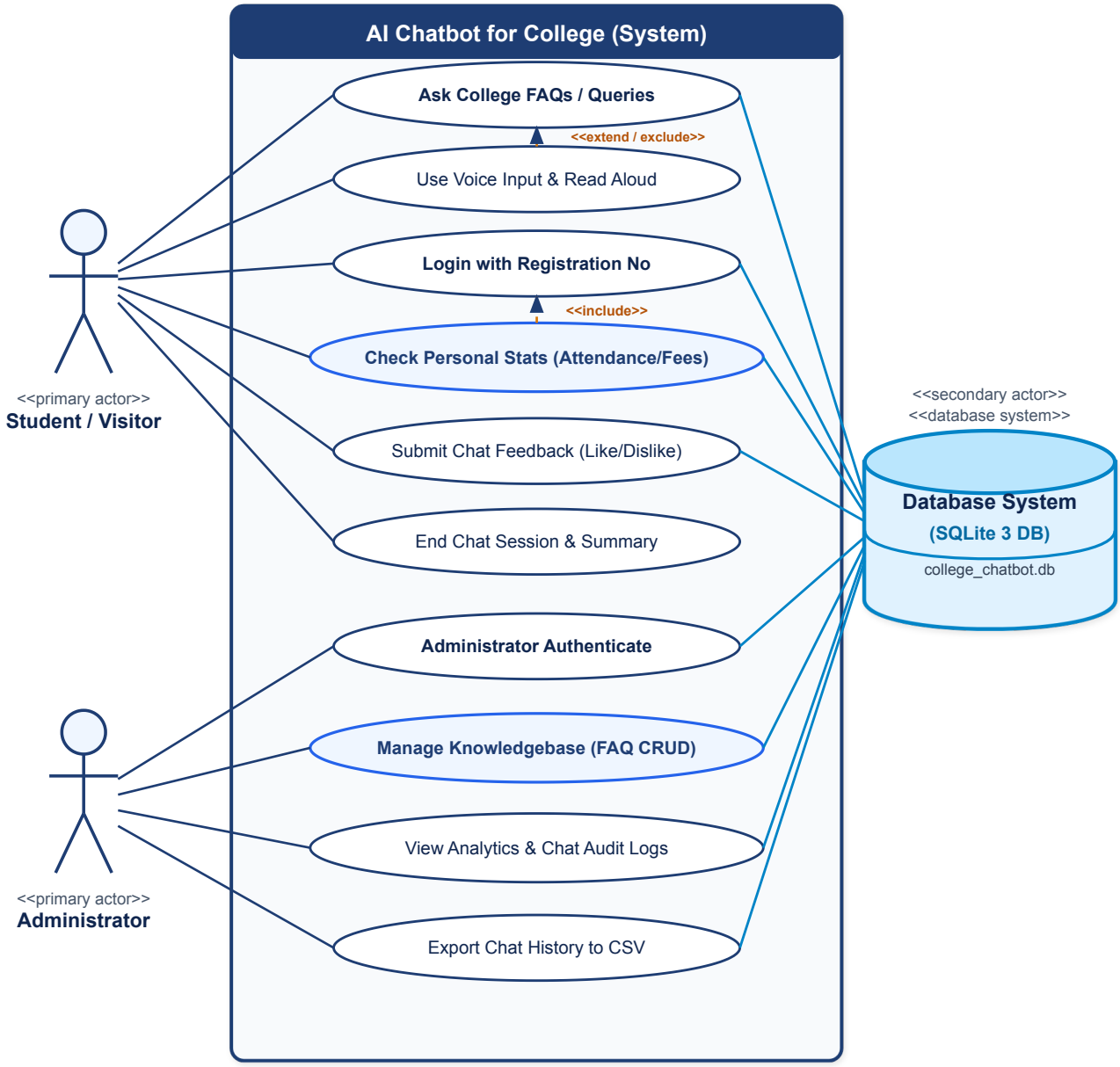


Figure 2.1: Complete Use Case Diagram with System Boundary, Database Cylinder, and Include/Extend Dependencies.

4. Draw UML Diagrams of your project — (b) Class Diagram

Object-Oriented Structure: Depicts the static class structure of the application. An abstract base class User provides common authentication credentials (username, password_hash, role) inherited via generalization (hollow triangle) by both Student and Admin. Structural associations specify exact multiplicities (1 Student submits 0..* ChatHistory logs; 1 Admin manages 0..* CollegeInfo FAQ entries; CollegeInfo is matched 0..1 in 0..* ChatHistory records).

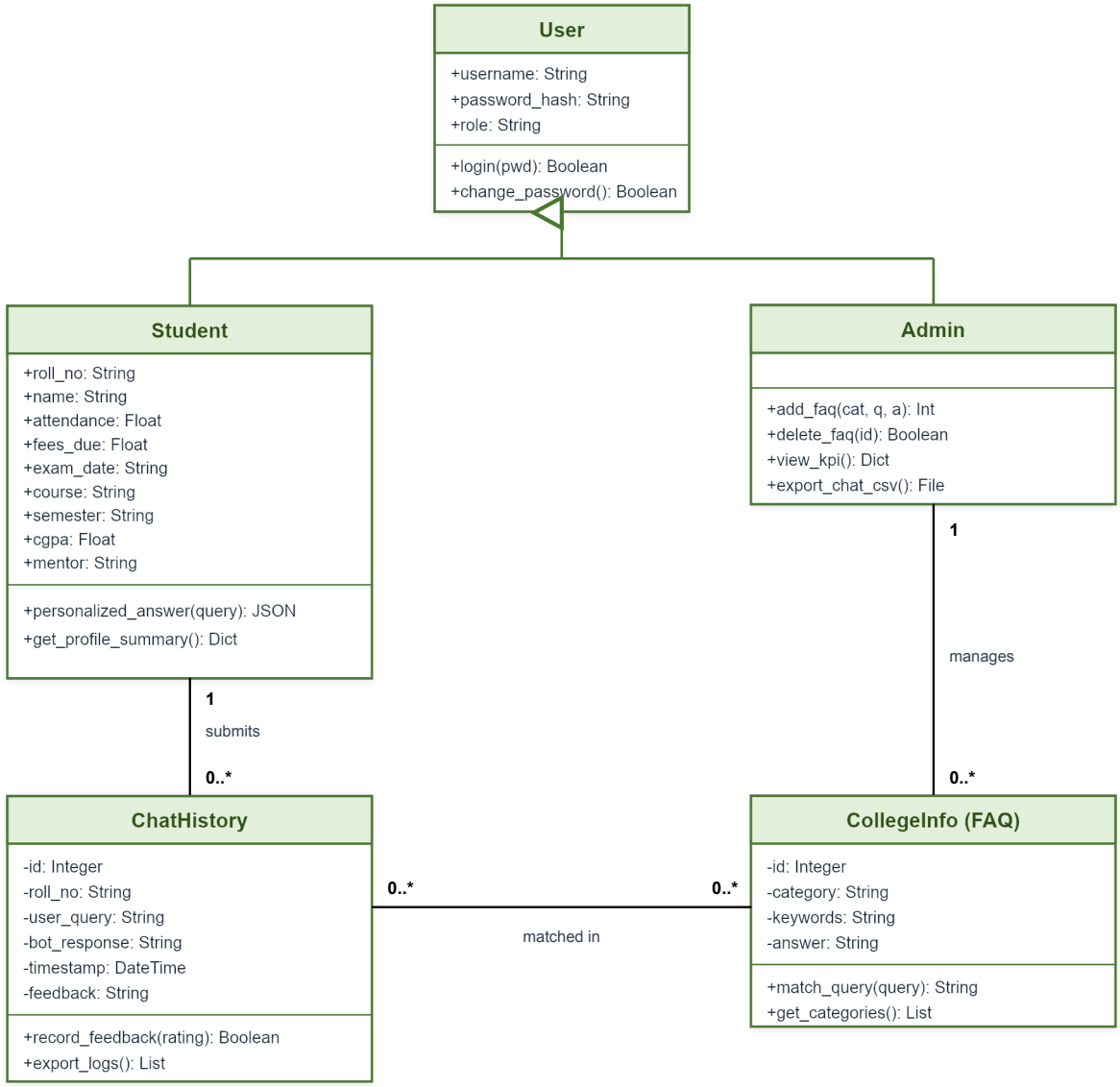


Figure 2.2: Standard Class Diagram with Attributes, Visibility Symbols (+/-), Methods, Inheritance & Multiplicities.

4. Draw UML Diagrams of your project — (c) Sequence Diagram

Temporal Message Sequencing: Illustrates the chronologically ordered exchange of messages during a student session. The sequence depicts lifelines for Student, Chatbot UI (Browser), Flask Server (app.py), NLP & Fuzzy Matcher, and SQLite Database (chatbot.db). It models synchronous request invocations (solid lines with arrows), database commits, asynchronous reply returns (dashed lines with open arrowheads), and feedback rating logging.

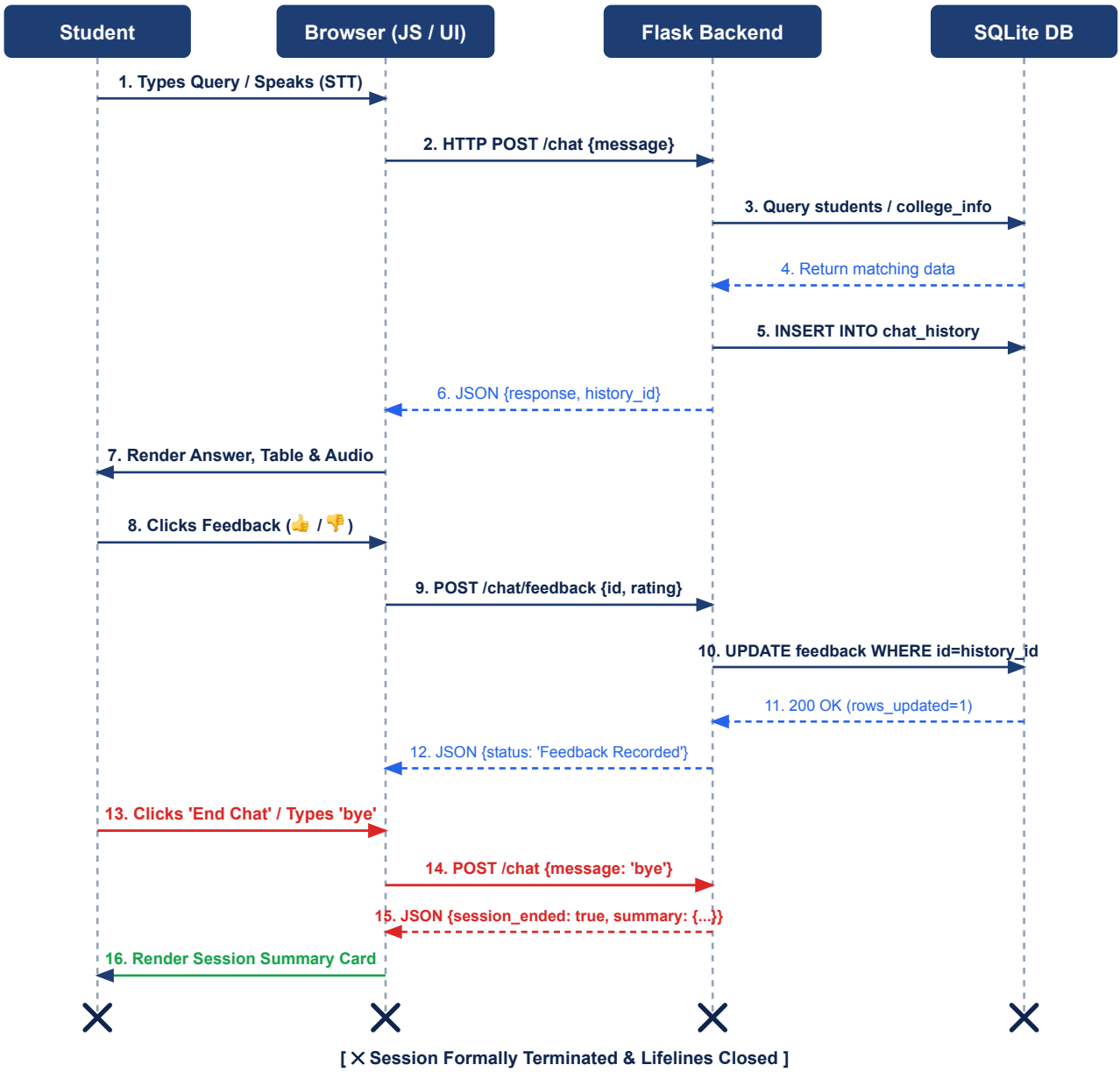


Figure 2.3: Sequence Diagram Modeling Message Flow, Activation Bars, Database Queries, and Audio Feedback.

4. Draw UML Diagrams of your project — (d) Activity Diagram

Workflow & Decision Logic: Traces the end-to-end execution flow of a user interaction. Starting from the initial node (filled circle), the activity captures user input, forks for optional Web Speech Speech-to-Text transcription, evaluates decision diamonds for student authentication status, checks 4-tier detention boundaries, runs fuzzy keyword algorithms, logs interaction records to SQLite, synthesizes spoken TTS audio, and terminates at the final state.

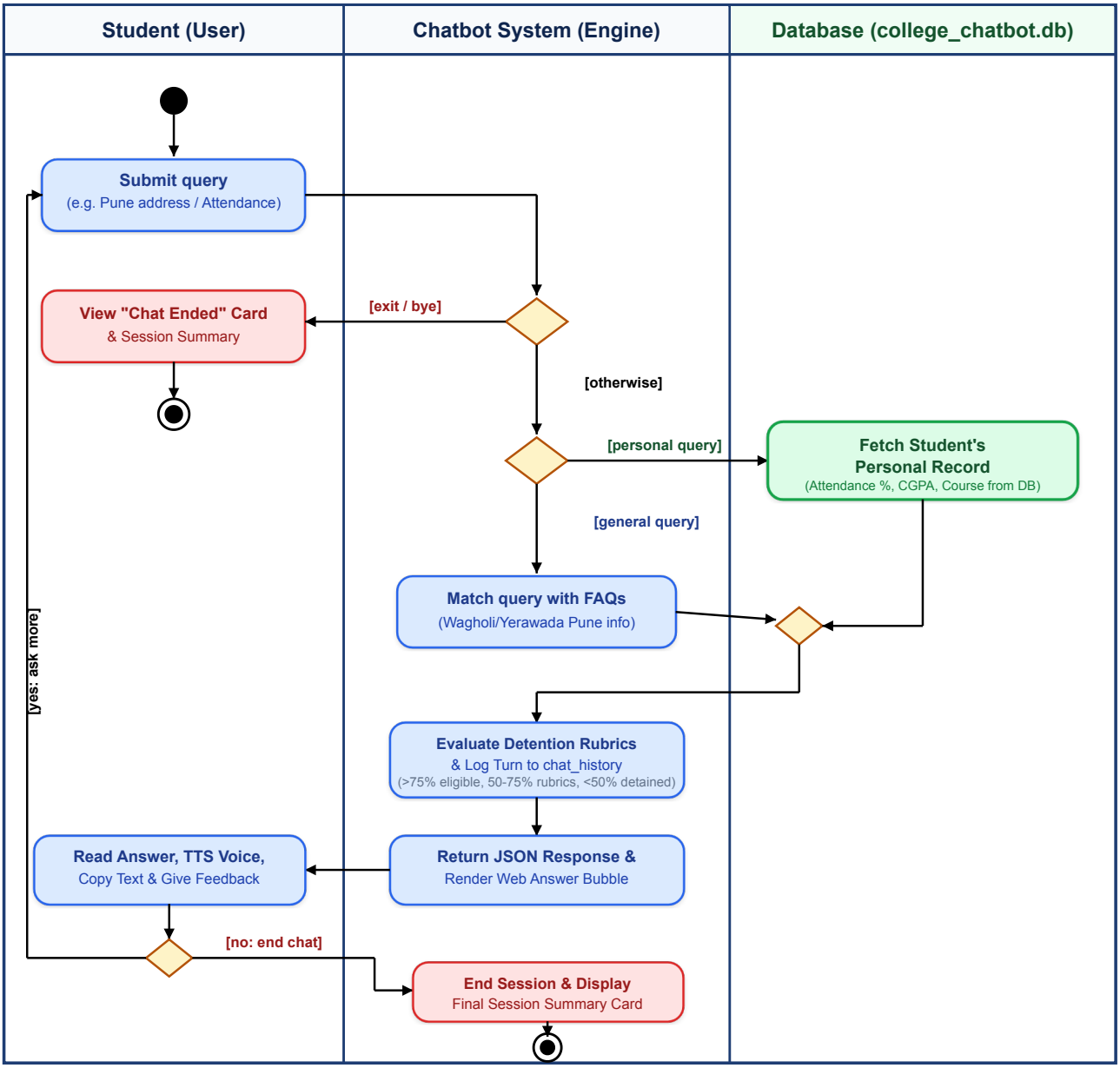
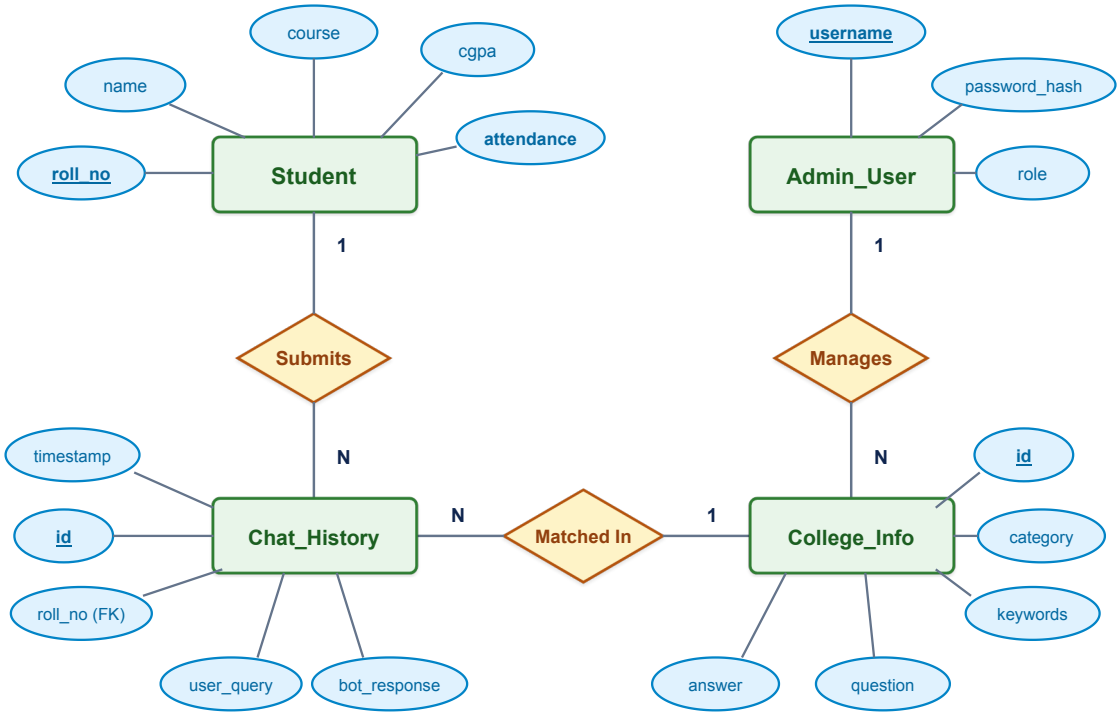


Figure 2.4: Activity Diagram Tracing Branching Logic, Intent Resolution, Detention Categorization & Termination.

4. Draw UML Diagrams of your project — (e) Component Diagram

Modular Component Architecture: Visualizes the structural relationships between autonomous software modules. The architecture illustrates three principal tiers: Client Presentation Tier (ChatbotUI), Application Controller Tier (FlaskServer hosting NLP_Matcher and Personalization Engine), and Data Persistence Tier (SQLite3 DB). External dependencies (Web Speech API and Google Text-to-Speech Engine) bind through standardized interface sockets.

ER Diagram (Classic Notation): AI Chatbot for College



Schema & Project Knowledge Base Defense:

- **Student Entity:** Stores attendance %, CGPA & evaluates 4-tier detention policy.
- **College_Info Entity:** Contains Pune campuses (Wagholi & Yerawada) & exam rules.
- **Chat_History Entity:** Logs each turn with student roll_no FK, query & bot answer.
- **Admin_User Entity:** RBAC authentication for updating college FAQs & detention data.

Verified in 3NF Normalization: Zero transitive dependencies & lossless join preservation.

Key / Legend:

- Entity (Table)
- Attribute (Column) — id = Primary Key
- Relationship
- Link / Cardinality (1:N)

Figure 2.5: Component Diagram Illustrating Multi-Tier Modular Wiring, Interface Sockets & Persistence Storage.